



Arquitecturas de IA

Arquitectura = diseño interno de la red. Como ingeniero no la implementas; la usas para **elegir** y **hablar con el equipo**.

Sin fórmulas: solo intuición visual.

1. Las tres que debes conocer

RNN

Secuencia
paso a paso
|
texto antiguo,
series cortas

CNN

Rejilla (imagen)
filtros locales
|
visión clásica
patrones en píxeles

TRANSFORMER

Secuencia / multimodal
"atención" global
|
LLM, embeddings,
multimodal hoy

2. RNN - Red Neuronal Recurrente

Idea visual: lee la secuencia **en orden**, llevando un "estado" de lo visto antes — como una memoria corta que se actualiza en cada paso.

$\text{palabra}_1 \rightarrow [\text{RNN}] \rightarrow \text{estado} \rightarrow \text{palabra}_2 \rightarrow [\text{RNN}] \rightarrow \text{estado} \rightarrow \dots$

Ventaja histórica

Limitación actual

Modelar secuencias (texto, series)

Difícil escalar a contextos muy largos

Simple de entender

Entrenamiento más lento (poco paralelismo)

Base de LSTM/GRU

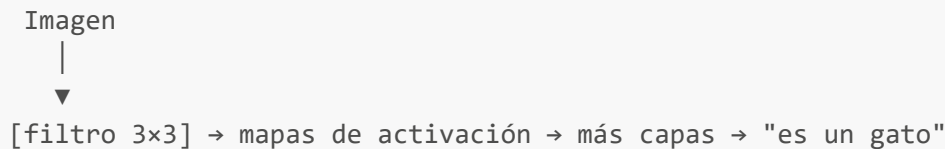
Superadas por Transformers en NLP y GenAI

Hoy en GenAI: raro como modelo principal de chat. Siguen apareciendo en **series temporales** o sistemas legacy.

Problema típico → RNN (histórico): predicción de serie corta, NLP clásico pre-2018.

3. CNN - red neuronal convolucional

Idea visual: desliza **filtros** pequeños sobre la imagen (o matriz) detectando bordes, texturas, formas — de lo local a lo global capa a capa.

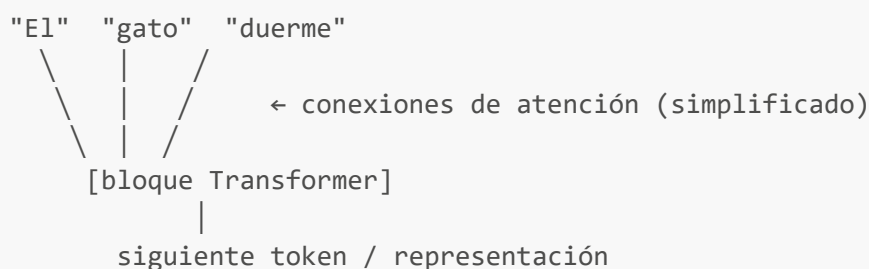


Ventaja	Limitación
Muy eficiente en imágenes	No nació para lenguaje largo
Clasificación, detección	Para chat largo no es la herramienta
Base de mucha visión clásica	ViT (Transformer) compite en visión

Problema típico → CNN: clasificación de imágenes, detección de objetos, visión industrial.

4. Transformer

Idea visual: cada token puede “mirar” a **todos** los demás a la vez (atención) — no hace falta recorrer la secuencia paso a paso como en RNN.



Qué interesa como ingeniero de IA a nivel conceptual:

- Trabaja con **tokens**, no con palabras sueltas.
- Escala bien en **hardware paralelo** (GPUs masivas).
- Soporta **contextos largos** (millones de tokens en modelos recientes).
- Base de **LLM**, muchos **embeddings** y modelos **multimodales**.

5. Por qué los Transformers dominan la IA generativa actual

Factor	RNN	CNN	Transformer
Texto largo y generación	Regular	No es su foco	Muy fuerte
Paralelizar entrenamiento	Difícil	Bueno en imágenes	Muy bueno
Un solo stack para varias tareas	Limitado	Visión	Texto, código, multimodal
Ecosistema (APIs, modelos)	Legacy	Visión + híbridos	Mayor oferta GenAI
Contexto amplio en producto	Corto	N/A	Ventanas grandes

Resumen en una frase: los Transformers combinan **escala, calidad en lenguaje** y **un ecosistema** que RNN y CNN solos no cubren para chat, código y multimodal — aunque CNN sigue siendo referencia en visión pura y RNN en nichos secuenciales.

6. Tabla problema → arquitectura → familia de modelos

Problema	Familia	Arquitectura típica
Chatbot	Texto (LLM)	Transformer
Búsqueda semántica	Embeddings	Transformer (encoder)
Clasificación imágenes	Visión	CNN (o ViT = Transformer)
OCR multimodal	Multimodal	Transformer multimodal
Detección objetos en vídeo	Visión	CNN + lógica temporal (a veces RNN/Transformer)
Predicción serie muy corta	Series	RNN/LSTM (menos común en GenAI)

7. Ventana de contexto. Consecuencia del Transformer en producto.

Los LLM Transformer tienen un **máximo de tokens** por llamada (input + salida).

Si tu caso...	Implicación
Chat corto	Casi cualquier modelo reciente vale
Documento largo	Necesitas ventana grande o trocear/resumir
RAG con muchos chunks	Suma tokens rápido → filtrar en Python

Tendremos que tener en cuenta **coste y medición** de esta ventana de contexto al desarrollar el producto.